Publica una página Angular en GitHub Pages

Manual paso a paso: del *Hola Mundo* al despliegue automático con GitHub Actions

Para quien ya programa, pero nunca ha publicado un sitio en GitHub Pages.

```
$ git push  ->  sitio publicado
```

Angular 21 | GitHub Actions | GitHub Pages

Contenido

1. Lo que vas a lograr **4**

Conceptos clave (en palabras simples) . . . 4

Por qué Angular hace esto más interesante . . . 5

2. Requisitos previos **5**

Instalar Angular CLI . . . 5

Autenticación con GitHub (solo si *push* te pide credenciales) . . . 5

3. Crear el proyecto Angular **6**

3.1 Generar la app dentro de la carpeta del repo . . . 6

3.2 Qué se creó . . . 6

3.3 Hacer el ¡Hola Mundo! . . . 7

3.4 Verlo en tu máquina . . . 8

4. Entender el build: salida y base href **9**

4.1 Compilar y mirar la carpeta de salida . . . 9

4.2 El problema del base href . . . 9

4.3 Dos formas de configurarlo . . . 10

4.4 Probar el build localmente (opcional pero recomendado) . . . 11

5. Subir el código a GitHub **11**

6. Activar GitHub Pages con GitHub Actions **12**

Pasos . . . 12

7. Crear el workflow de despliegue **14**

7.1 Explicación, bloque por bloque . . . 16

7.2 Lo único que debes personalizar . . . 16

8. Tu primer despliegue **17**

Seguir el progreso . . . 17

9. El ciclo de trabajo diario **18**

Prueba que sea automático . . . 18

Trabajar con ramas (buena práctica) . . . 18

10. Extra: rutas de Angular (router) en Pages **19**

Solución 1 (recomendada): copiar index.html como 404.html 19

Solución 2: rutas con hash 19

11. Problemas comunes y cómo resolverlos **20**

12. Resumen, lista de verificación y glosario **21**

Lista de verificación 21

Chuleta de comandos 21

Glosario 22

1 Lo que vas a lograr

Al terminar este manual tendrás una página **¡Hola Mundo!** hecha con Angular, publicada en internet gratis con **GitHub Pages**, y configurada para que **cada vez que hagas `git push` a la rama main se vuelva a compilar y publicar sola**. Nada de subir archivos a mano.

Conceptos clave (en palabras simples)

Concepto	Qué es
GitHub Pages	Servicio gratuito de GitHub que hospeda **sitios estáticos** (HTML, CSS y JavaScript ya listos). No ejecuta PHP, Node ni bases de datos.
Sitio estático	Archivos que el navegador descarga tal cual. Angular, al compilar, produce exactamente eso: una carpeta con index.html + .js + .css.
Build (compilar)	Convertir tu código TypeScript/Angular en archivos optimizados que cualquier navegador entiende. Se hace con `ng build`.
Carpeta de salida	Donde Angular deja el resultado del build: `dist/hola-mundo/browser`. **Esa carpeta es la que se publica**, no tu código fuente.
CI/CD	Integración y Entrega Continuas: un robot que, ante cada cambio, instala, compila y publica por ti.
GitHub Actions	El "robot" de GitHub. Lee un archivo YAML (workflow) con las instrucciones y las ejecuta en una máquina virtual temporal.
Workflow	Archivo `.github/workflows/*.yml` que describe cuándo correr y que pasos seguir.

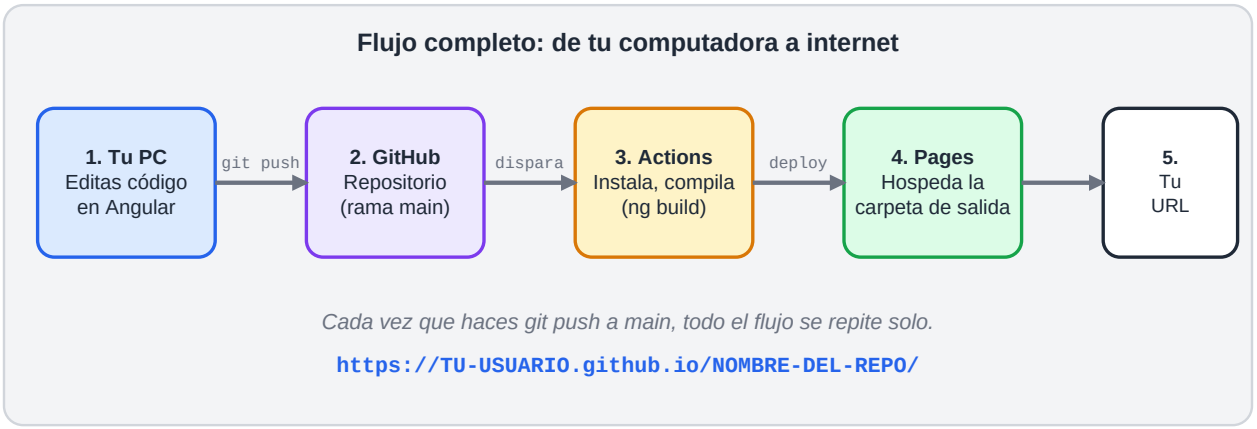


Figura 1. Vista general de lo que construiremos

Por qué Angular hace esto más interesante

Un HTML suelto se puede subir tal cual. Angular no: necesita un **paso de compilación**. Por eso aprenderás tres cosas que aplican a cualquier framework (React, Vue, Svelte...): **(1)** cuál comando compila, **(2)** en qué carpeta queda el resultado y **(3)** cómo decirle al sitio que vive bajo una sub-ruta (*base href*).

2 Requisitos previos

Necesitas lo siguiente instalado. Cada comando de verificación debe mostrar una versión (los números exactos pueden variar).

Herramienta	Para qué	Verificar con
Cuenta de GitHub	Alojar el repo y publicar	github.com (gratis)
Git	Subir cambios al repo	`git --version`
Node.js 20.19+ / 22.12+ / 24	Ejecutar Angular CLI	`node -v`
npm (viene con Node)	Instalar dependencias	`npm -v`
Angular CLI	Crear, servir y compilar	`ng version`
Un editor (VS Code)	Escribir código	-

Instalar Angular CLI

terminal

```
npm install -g @angular/cli
```

Si ya la tienes, `ng version` te muestra la versión. Este manual se verificó con **Angular CLI 21** y **Node 22**.

> **Tu repositorio ya existe**
> En este proyecto el repo ya está creado en GitHub y la carpeta local está conectada (`origin`). Comprueba con `git remote -v`; debe mostrar la URL de tu repo. Usaremos **learn_github_pages_ci_cd** como ejemplo: reemplaza ese nombre y tu usuario por los tuyos si son distintos.

Autenticación con GitHub (solo si *push* te pide credenciales)

GitHub ya no acepta tu contraseña en `git push`. Elige una opción:

- **GitHub CLI** (la más fácil): instala gh y ejecuta `gh auth login`.

- **Token personal**: GitHub > Settings > Developer settings > Personal access tokens. Ese token se usa como contraseña.
- **SSH**: genera una llave y cambia el remoto a la URL SSH.

3 Crear el proyecto Angular

3.1 Generar la app dentro de la carpeta del repo

Abre la terminal **dentro de la carpeta de tu repositorio** y ejecuta. El `--directory .` hace que Angular cree los archivos **aquí mismo** (en la raíz) en lugar de crear una subcarpeta:

`terminal`

```
ng new hola-mundo --directory . --defaults --skip-git
```

Opción	Significado
`hola-mundo`	Nombre del proyecto. Importante: define el nombre de la carpeta de salida (`dist/hola-mundo`).
`--directory .`	Crear en la carpeta actual (la raíz del repo).
`--defaults`	Acepta opciones por defecto (CSS, sin SSR, sin preguntas).
`--skip-git`	No ejecutar *git init*: el repo ya existe.

> **¿Salió el error "Cannot read properties of null (reading 'edgesOut')"?**
> Es un bug conocido de npm 10 al instalar dependencias de Angular 21. La app se crea pero la instalación falla. Solución: crea un archivo llamado `.npmrc` en la raíz con la línea `legacy-peer-deps=true` y luego ejecuta `npm install`. Ese archivo **también lo usara GitHub Actions**, así que súbelo al repo. Si no te dio error, no lo necesitas.

`terminal (solo si fallo la instalación)`

```
echo "legacy-peer-deps=true" > .npmrc
npm install
```

3.2 Qué se creó

Estos son los archivos y carpetas que importan:

estructura del proyecto

```
learn_github_pages_ci_cd/
|-- angular.json          <- configuración del build (carpeta de salida, etc.)
|-- package.json          <- dependencias y scripts (npm run build)
|-- package-lock.json     <- versiones exactas (lo usa npm ci en CI)
|-- tsconfig*.json        <- configuración de TypeScript
|-- public/               <- archivos estáticos que se copian tal cual (favicon)
|-- src/
|   |-- index.html        <- página base (aquí vive <base href>)
|   |-- main.ts           <- punto de entrada que arranca la app
|   |-- styles.css        <- estilos globales
|   `-- app/
|       |-- app.ts        <- componente raíz (lógica)
|       |-- app.html      <- plantilla (lo que se ve)
|       |-- app.css       <- estilos del componente
|       `-- app.config.ts <- configuración de la aplicación
`-- (node_modules/ y dist/ se generan solos y NO se suben a git)
```

3.3 Hacer el ¡Hola Mundo!

Angular genera una página de bienvenida enorme. La reemplazaremos por algo mínimo. Edita estos tres archivos:

src/app/app.ts

```
import { Component } from '@angular/core';
@Component({
  selector: 'app-root',
  templateUrl: './app.html',
  styleUrl: './app.css',
})
export class App {
  protected readonly titulo = '¡Hola Mundo!';
  protected readonly anio = new Date().getFullYear();
}
```

src/app/app.html

```
<main class="tarjeta">
  <h1>{{ titulo }}</h1>
  <p>Mi primera página en Angular, desplegada con GitHub Pages.</p>
  <small>Desplegada automáticamente con GitHub Actions - {{ anio }}</small>
</main>
```

`src/app/app.css`

```
:host {
  display: grid;
  place-items: center;
  min-height: 100vh;
  font-family: system-ui, sans-serif;
  background: #0f172a;
  color: #f8fafc;
}
.tarjeta { text-align: center; }
h1 { font-size: 3rem; margin: 0 0 0.5rem; }
small { opacity: 0.6; }
```

Los `{{ }}` son **interpolación**: Angular reemplaza `{{ titulo }}` por el valor de la propiedad `titulo` de la clase. Así comprobamos que Angular realmente funciona en el sitio publicado y no es un HTML estático.

> **Sobre el archivo app.spec.ts**
> Angular crea una prueba de ejemplo que busca el texto "Hello, hola-mundo". Como cambiamos el título, `ng test` fallaría. Este manual no ejecuta pruebas en el pipeline, pero si quieres mantenerlas verdes cambia ese texto por ¡Hola Mundo! en la línea del `expect`.

3.4 Verlo en tu máquina

`terminal`

```
ng serve
```

Abre **http://localhost:4200**. Verás tu ¡Hola Mundo!. El servidor recarga solo al guardar cambios. Detenlo con `Ctrl + C`.

4 Entender el build: salida y base href

Este es el capítulo más importante del manual. `ng serve` es solo para desarrollo. Lo que se publica es el resultado de `ng build`.

4.1 Compilar y mirar la carpeta de salida

terminal

```
ng build
```

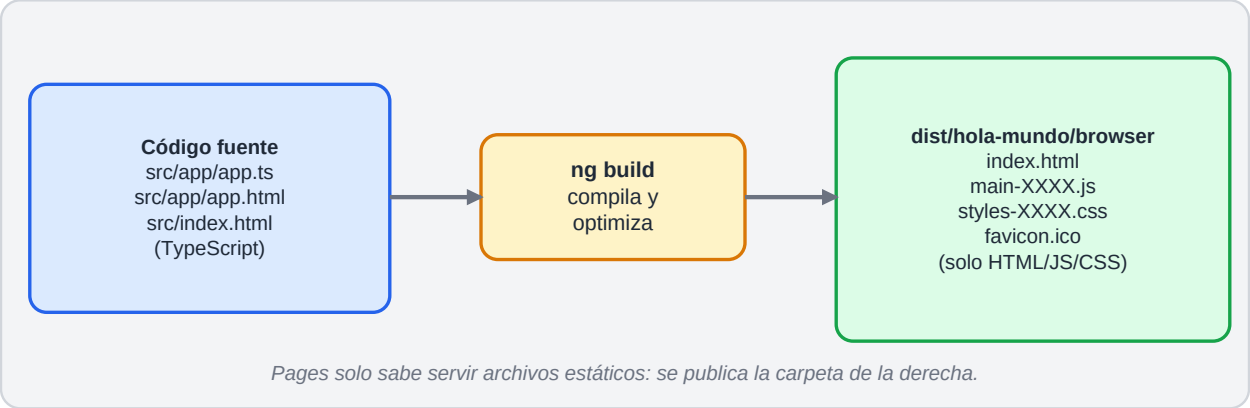


Figura 2. El build transforma el código fuente en archivos estáticos

Desde Angular 17, la salida queda en `dist/<nombre-del-proyecto>/browser`. La subcarpeta `browser` contiene lo que va al navegador. Si tu proyecto se llamara *mi-app*, la ruta sería `dist/mi-app/browser`.

> **Cómo saber tu carpeta de salida sin adivinar**
> Al terminar, `ng build` imprime **Output location**. También puedes revisarla en `angular.json`: la clave del proyecto (dentro de "projects") es el nombre, y el builder `@angular/build:application` usa por defecto `dist/<nombre>` + `/browser`.

4.2 El problema del base href

Un repositorio normal (no el especial *usuario.github.io*) se publica en una **sub-ruta**, no en la raíz del dominio:



Figura 3. La URL contiene el nombre del repo

Mira `src/index.html`: trae `<base href="/">`. Eso le dice al navegador que busque `main.js` en `https://vlv-515.github.io/main.js`, pero el archivo realmente está en `https://vlv-515.github.io/learn_github_pages_ci_cd/main.js`. Resultado: **página en blanco y errores 404 en la consola**.

Solución: compilar indicando el nombre del repo (con barra al inicio **y al final**):

`terminal`

```
ng build --base-href /learn_github_pages_ci_cd/
```

El build reescribe la etiqueta en el index.html de salida:

`dist/hola-mundo/browser/index.html (resultado)`

```
<base href="/learn_github_pages_ci_cd/">
```

Caso	Valor de --base-href
Repo normal: usuario.github.io/**mi-repo**	`/mi-repo/`
Repo especial: **usuario.github.io** (raíz)	`/` (no hace falta nada)
Dominio propio (ej. midominio.com)	`/`

4.3 Dos formas de configurarlo

Opción A (la que usaremos): bandera en el comando de CI. El comando local `ng build` sigue sirviendo para probar y solo en la nube se agrega el base href. Es explicito y fácil de entender.

Opción B: dejarlo en angular.json, dentro de la configuración de producción. Entonces `ng build` ya lo trae siempre:

`angular.json (fragmento, opcional)`

```
"build": {
  "builder": "@angular/build:application",
  "options": { ... },
  "configurations": {
    "production": {
      "baseHref": "/learn_github_pages_ci_cd/",
      ...
    }
  }
}
```

4.4 Probar el build localmente (opcional pero recomendado)

Para ver la salida real antes de subirla, compila **sin** base href y sirve la carpeta:

`terminal`

```
ng build
npx http-server dist/hola-mundo/browser -p 8080
```

Abre **http://localhost:8080**. Si ves tu ¡Hola Mundo!, el build es correcto. (Si lo compilaras con el base href del repo, localmente se vería en blanco porque la sub-ruta no existe aquí; es normal.)

5 Subir el código a GitHub

Primero crea el archivo `.gitignore`; Angular ya lo genero e ignora `node_modules/` y `dist/`. Eso es correcto: la carpeta compilada **no se sube**, la genera GitHub Actions.

Revisa el estado y haz tu primer commit del proyecto:

`terminal`

```
git status
git add .
git commit -m "feat: proyecto Angular hola mundo"
git branch -M main
git push -u origin main
```

Comando	Qué hace
`git status`	Lista los archivos nuevos o modificados.
`git add .`	Marca todos los cambios para el siguiente commit.
`git commit -m "..."`	Guarda una "foto" de los cambios con un mensaje.
`git branch -M main`	Asegura que la rama principal se llame main.

Comando	Qué hace
`git push -u origin main`	Envia a GitHub. El *-u* recuerda el destino para futuros `git push` sin argumentos.

Refresca la página de tu repo en GitHub: debes ver `src/`, `angular.json`, etc. En este punto **aún no hay sitio publicado**; falta activar Pages y crear el workflow.

6 Activar GitHub Pages con GitHub Actions

Hay dos formas de publicar en Pages:

Modo	Cómo funciona	Cuándo usarlo
Deploy from a branch	Publica archivos que ya están en una rama (ej. gh-pages o /docs).	Sitios sin build (HTML puro).
GitHub Actions	Un workflow compila y entrega el resultado a Pages. **No necesitas rama gh-pages.**	**Angular, React, Vue...** (es lo moderno y lo que usamos).

Pasos

1. En GitHub abre tu repositorio y entra a la pestaña **Settings** (engrane, arriba a la derecha).
2. En el menu lateral izquierdo, sección *Code and automation*, haz clic en **Pages**.
3. En **Build and deployment > Source**, abre el desplegable y elige **GitHub Actions**.
4. No hay botón de guardar: el cambio se aplica al seleccionar. No aparece nada más por ahora; es normal.

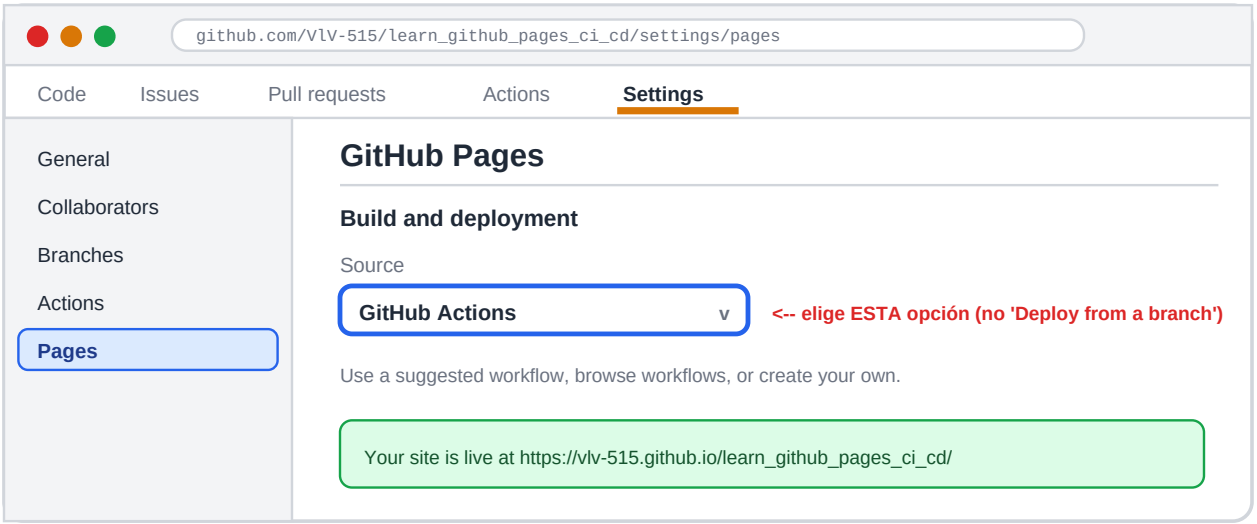


Figura 4. Settings > Pages > Source = GitHub Actions (maqueta ilustrativa)

Repos privados
En el plan gratuito de GitHub, Pages solo está disponible para **repositorios públicos**. Si tu repo es privado y no ves la opción, hazlo público (Settings > General > Danger Zone > Change visibility) o usa un plan de pago.

7 Crear el workflow de despliegue

`.github/workflows/deploy.yml (crea este archivo con este contenido exacto)`

```yaml
name: Deploy a GitHub Pages
on:
  push:
    branches: [main]
  workflow_dispatch:
permissions:
  contents: read
  pages: write
  id-token: write
concurrency:
  group: pages
  cancel-in-progress: true
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Descargar código
        uses: actions/checkout@v4
      - name: Instalar Node
        uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - name: Instalar dependencias
        run: npm ci
      - name: Compilar Angular
        run: npm run build -- --base-href /${{ github.event.repository.name }}/
      - name: Empaquetar para Pages
        uses: actions/upload-pages-artifact@v3
        with:
          path: dist/hola-mundo/browser
  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    steps:
      - name: Publicar
        id: deployment
        uses: actions/deploy-pages@v4
```

Los workflows viven en una ruta **exacta**: `.github/workflows/` (con punto al inicio) en la raíz del repo. Crea las carpetas y el archivo `deploy.yml` con el contenido de arriba.

YAML y los espacios
YAML usa **espacios para la estructura** (2 por nivel). Nunca tabuladores. Un espacio mal puesto rompe todo. Copia el archivo tal cual y cambia solo lo que se indica abajo.

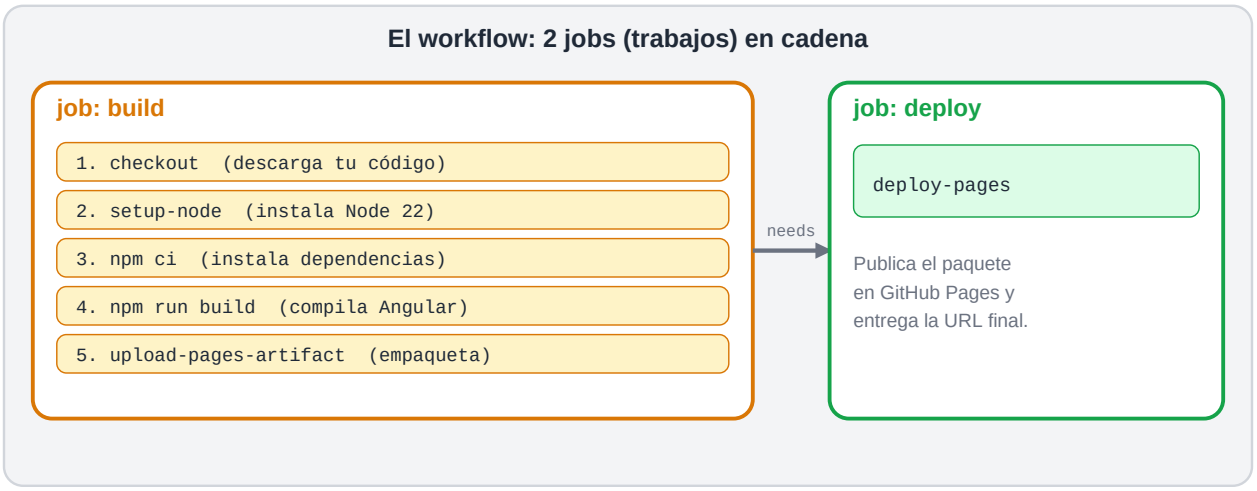


Figura 5. Estructura del workflow

7.1 Explicación, bloque por bloque

Bloque	Qué hace y por qué
`name`	Nombre visible en la pestaña Actions.
`on.push.branches: [main]`	**El disparador.** Corre cada vez que hay push a main. Es lo que da el despliegue automático.
`on.workflow_dispatch`	Agrega un botón "Run workflow" para lanzarlo manualmente. Útil para reintentar.
`permissions`	Permisos minimos del robot: leer el código (*contents*), escribir en Pages (*pages*) y obtener un token de identidad (*id-token*) para verificar el despliegue.
`concurrency`	Si haces dos push seguidos, cancela el despliegue viejo y publica solo el último.
`jobs.build`	Primer trabajo: compila. `runs-on: ubuntu-latest` es la máquina virtual Linux.
`actions/checkout`	Descarga tu repo en la máquina virtual (empieza vacia).
`actions/setup-node`	Instala Node 22. `cache: npm` guarda descargas para que corra más rápido.
`npm ci`	Instala dependencias exactamente según package-lock.json (más estricto y reproducible que *npm install*).
`npm run build -- --base-href ...`	Ejecuta el script *build* de package.json (*ng build*). El `--` pasa lo que sigue a ng. `${{ github.event.repository.name }}` se sustituye por el nombre del repo, así no lo escribes a mano.
`upload-pages-artifact`	Empaqueta la carpeta indicada en *path*. **Debe coincidir con tu carpeta de salida.**
`jobs.deploy`	Segundo trabajo. `needs: build` = espera a que build termine bien.
`environment github-pages`	Entorno requerido por Pages. Expone la URL final en *page_url*.
`actions/deploy-pages`	Entrega el paquete a GitHub Pages: el sitio queda publicado.

7.2 Lo único que debes personalizar

- `path: dist/**hola-mundo**/browser`: cambia *hola-mundo* si tu proyecto tiene otro nombre (el que pusiste en `ng new`).
- `node-version`: mantenlo igual o superior a la que pide tu Angular CLI.
- La rama `main`: cambiala si tu rama principal se llama distinto (ej. master).

8 Tu primer despliegue

Sube el workflow (y el .npmrc si lo creaste). Este push **es el disparador**:

terminal

```
git add .
git commit -m "ci: workflow de despliegue a GitHub Pages"
git push
```

Seguir el progreso

1. En GitHub, abre la pestaña **Actions**.
2. Verás una ejecución nueva con circulo amarillo (en curso). Haz clic para ver los jobs.
3. Primero corre *build*, luego *deploy*. Toma 1 a 2 minutos.
4. Al terminar, el circulo se pone verde. En el job *deploy* aparece el enlace del sitio.

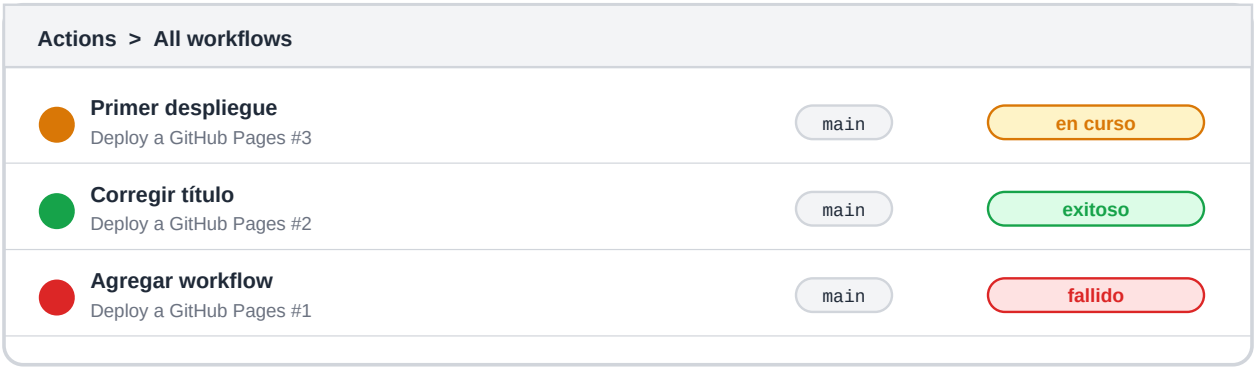


Figura 6. La pestaña Actions: en curso (amarillo), exitoso (verde), fallido (rojo)

Tu sitio queda en:

```
https://vlv-515.github.io/learn_github_pages_ci_cd/
```

También lo encuentras en **Settings > Pages** ("Your site is live at..."). Recuerda: la URL usa tu usuario en minúsculas. ¡Ábrela y verás tu ¡Hola Mundo!!

> **Primera vez, ¿sale 404?**
> Espera 1 o 2 minutos después del check verde y recarga con `Ctrl + Shift + R` (sin caché). Si sigue, revisa el capítulo de problemas comunes.

9 El ciclo de trabajo diario

Esto es lo que querías lograr: el despliegue ya es automático. Para cambiar el sitio:



Figura 7. Flujo diario

1. Edita tu código (por ejemplo cambia el texto en `app.html`).
2. Prueba local con `ng serve`.
3. `git add .` y `git commit -m "fix: cambio de texto"`.
4. `git push`. Listo: Actions compila y publica.
5. En 1 a 2 minutos recarga la URL y verás el cambio.

Prueba que sea automático

`terminal`

```
# 1. cambia el h1 en src/app/app.html, por ejemplo:
#    <h1>{{ titulo }} v2</h1>
git add .
git commit -m "feat: versión 2"
git push
# 2. mira la pestaña Actions y luego recarga tu sitio
```

Trabajar con ramas (buena práctica)

El workflow solo despliega cuando hay push a **main**. Puedes trabajar en una rama aparte (`git switch -c mi-cambio`), subirla y abrir un *Pull Request*: no se publica nada hasta que lo fusiones a main. Así pruebas con calma y publicas solo lo aprobado.

10 Extra: rutas de Angular (router) en Pages

Tu Hola Mundo tiene una sola página, pero cuando agregues rutas (`/about`, `/contacto`) aparecerán un problema: si alguien **recarga** o entra directo a `.../learn_github_pages_ci_cd/about`, GitHub Pages busca un archivo *about*, no lo encuentra y devuelve **404**. Angular es una SPA: todo vive en index.html.

Solución 1 (recomendada): copiar index.html como 404.html

Pages sirve `404.html` cuando no encuentra una ruta. Si ese archivo es una copia de index.html, Angular arranca y resuelve la ruta. Agrega este paso en el job *build*, justo después de compilar:

deploy.yml (paso adicional, después de 'Compilar Angular')

```
      - name: Soporte de rutas SPA
        run: cp dist/hola-mundo/browser/index.html dist/hola-mundo/browser/404.html
```

Solución 2: rutas con hash

Cambia las URL a `.../#/about`. Funciona sin trucos porque el servidor nunca ve lo que va después del #. En `app.config.ts` usa `provideRouter(routes, withHashLocation())`. Es menos bonito, pero es a prueba de fallos.

> **Nota**
> Con la solución 1 el navegador recibe un código 404 aunque la página se muestre bien. Para proyectos personales no importa; para SEO serio considera otro hosting.

11 Problemas comunes y cómo resolverlos

Cuando algo falle, ve a **Actions**, abre la ejecución en rojo, haz clic en el job que falló y expande el paso con la X roja. El mensaje de error casi siempre dice la causa.

Síntoma	Causa probable	Solución
Sitio en blanco; consola con 404 de main-XXXX.js	Falta el base href.	Compilar con `--base-href /nombre-del-repo/` (con barras al inicio y al final). Mayúsculas/minúsculas importan.
404 "There isn't a GitHub Pages site here"	Pages no está activado, o aún no termina el primer despliegue.	Settings > Pages > Source = GitHub Actions. Espera el check verde y 1-2 min.
Error "Get Pages site failed" / "Not Found" en configure o deploy	Source no está en GitHub Actions.	Cambia el Source (capítulo 6) y vuelve a ejecutar: Actions > el workflow > Re-run all jobs.
Error "Resource not accessible by integration"	Faltan permisos en el workflow.	Verifica el bloque `permissions` (pages: write, id-token: write).
`npm ci` falla: "package.json and package-lock.json are not in sync" o error de peer deps	Lockfile generado distinto al de CI (bug de npm 10).	Crea `.npmrc` con `legacy-peer-deps=true`, borra node_modules y package-lock.json, ejecuta `npm install`, y sube ambos archivos.
Error "No files were found with the provided path: dist/..."	El *path* de upload-pages-artifact no coincide con la salida.	Revisa "Output location" en el log del build y corrige el path (incluye /browser).
El workflow no aparece o no se ejecuta	Ruta o extensión incorrecta; rama distinta de main.	Debe ser `.github/workflows/deploy.yml`. Verifica la rama en *on.push.branches*.
Subí un cambio y el sitio sigue igual	Caché del navegador, o el workflow aún corre.	Revisa Actions; recarga con Ctrl+Shift+R.
Build falla por "budget exceeded"	Un archivo supera los límites de tamaño de angular.json.	Reduce el tamaño o ajusta *budgets* en angular.json.
"Environment protection rules" bloquea el deploy	El entorno github-pages tiene reglas de rama.	Settings > Environments > github-pages: permite la rama main.

12 Resumen, lista de verificación y glosario

Lista de verificación

Marca cada punto. Si todos están cumplidos, el despliegue funciona.

[ ]	Angular CLI instalado y proyecto creado con `ng new`
[ ]	`ng serve` muestra el ¡Hola Mundo! en localhost:4200
[ ]	`ng build` genera `dist/hola-mundo/browser`
[ ]	Código subido a la rama **main** del repo en GitHub
[ ]	Settings > Pages > Source = **GitHub Actions**
[ ]	Existe `.github/workflows/deploy.yml` con el path de salida correcto
[ ]	El workflow pasa en verde en la pestaña Actions
[ ]	La URL `https://usuario.github.io/repo/` muestra tu página
[ ]	Un cambio nuevo + `git push` se publica solo

Chuleta de comandos

Comando	Para qué
`ng new nombre --directory .`	Crear proyecto en la carpeta actual
`ng serve`	Servidor de desarrollo (localhost:4200)
`ng build`	Compilar para producción (carpeta dist/)
`ng build --base-href /repo/`	Compilar para sub-ruta de GitHub Pages
`npm ci`	Instalar dependencias exactas (CI)
`git add . && git commit -m "msg" && git push`	Guardar y desplegar un cambio

Glosario

Término	Significado
Artifact	Paquete de archivos que un job entrega al siguiente (aquí, tu sitio compilado).
base href	Etiqueta HTML que define la ruta base desde la que se cargan los recursos.
Job	Grupo de pasos que corre en una misma máquina virtual.
Runner	La máquina virtual donde corre un job (*ubuntu-latest*).
SPA	Single Page Application: una sola página HTML; el router de Angular cambia la vista.
Step	Un paso individual dentro de un job: ejecutar un comando o usar una action.
Action	Pieza reutilizable creada por GitHub o la comunidad (*uses: ...*).
Workflow	Archivo YAML con la automatización completa.

Siguientes pasos
Agrega rutas y componentes, ejecuta `ng test` como paso extra antes del build, conecta un dominio propio (Settings > Pages > Custom domain), o despliega un segundo proyecto repitiendo este mismo proceso.